



RAPPORT DE PROJET : DATABASE

Subject :

Rapport d'évaluation de l'équipe de sécurité
sur le projet Oued Souss Alerte

Réalisé par :

Ait El Hadj Mohamed
AHANNACH EL M'SIRI OUIAM

Filière : SITCN

Groupe : 1

12 avril 2026

Résumé exécutif

Synthèse des vulnérabilités

Vulnérabilité	Criticité	Impact	Action prioritaire
Absence d'authentification (Broken Access Control)	CRITIQUE	Modification, suppression de toutes les données	Implémenter JWT ou API Key immédiatement
CORS permissif	ÉLEVÉ	Exposition aux attaques cross-origin	Restreindre aux origines de confiance
Absence de headers de sécurité	ÉLEVÉ	Vulnérabilités XSS, clickjacking	Ajouter Helmet
Absence de rate limiting	ÉLEVÉ	Risque de DoS et brute force	Mettre en place express-rate-limit
Journalisation excessive	MOYEN	Fuite d'informations sensibles dans les logs	Masquer les données sensibles

TABLE 1 – Synthèse des vulnérabilités majeures

Recommandations immédiates

RECOMMANDATION

Actions critiques à mener avant toute mise en production :

1. **Ajouter une authentification** sur toutes les routes d'écriture et d'administration.
2. **Restreindre CORS** à une liste blanche d'origines autorisées.
3. **Durcir Express** avec Helmet, rate limiting et désactivation de l'en-tête X-Powered-By.
4. **Réduire la verbosité des logs** pour éviter les fuites de données.

Table des matières

Résumé exécutif	1
Table de figures	3
Table des abréviations	4
Introduction	5
1 Méthodologie de travail	6
1.1 Revue du code (Code Review)	6
1.2 Vérification des mécanismes d'authentification	7
1.3 Analyse des risques (Risk Analysis)	7
1.4 Tests d'intrusion internes (Pentesting)	7
1.5 Corrélation et validation des résultats	8
1.6 Recommandations de sécurité	8
Version IA	9
1. Analyse de l'architecture sécurisée	9
2. Identification des mécanismes de sécurité	9
3. Analyse des bonnes pratiques de développement sécurisé	9
Version manuelle	10
1. Introduction + résumé exécutif	10
2. Vulnérabilités détectées (priorité critique)	10
3. Corrections proposées	11
4. Tableau comparatif AVANT / APRÈS	12
5. Bonnes pratiques (OWASP)	13
6. Conclusion partielle	13
Comparaison	14
Conclusion	15
Bibliographie	16

Table des figures

1	Carte de la région du Souss et emplacement des stations de mesure	5
2	Démarche méthodologique de l'audit de sécurité	6
3	Résultats du scan OWASP ZAP sur l'API	7
4	Architecture sécurisée en trois couches identifiée par l'IA	9
5	Flux des requêtes sans authentification - accès public aux endpoints critiques	10
6	Architecture de sécurité avant et après corrections	13
7	Comparaison radar des scores de sécurité (IA vs Manuel)	14
8	Plan d'action recommandé pour la sécurisation	15



Table des abréviations

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
DoS	Denial of Service
JWT	JSON Web Token
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
SIEM	Security Information and Event Management
XSS	Cross-Site Scripting
ZAP	Zed Attack Proxy

Introduction

Dans un contexte où les systèmes d'information jouent un rôle essentiel dans la gestion et la surveillance des ressources naturelles, la sécurité des bases de données devient un enjeu majeur. Le projet **Oued Souss Alerte** s'inscrit dans cette dynamique en proposant une solution de suivi et d'alerte liée aux niveaux d'eau dans la région du Souss. Ce type d'application repose fortement sur l'intégrité, la disponibilité et la confidentialité des données manipulées.

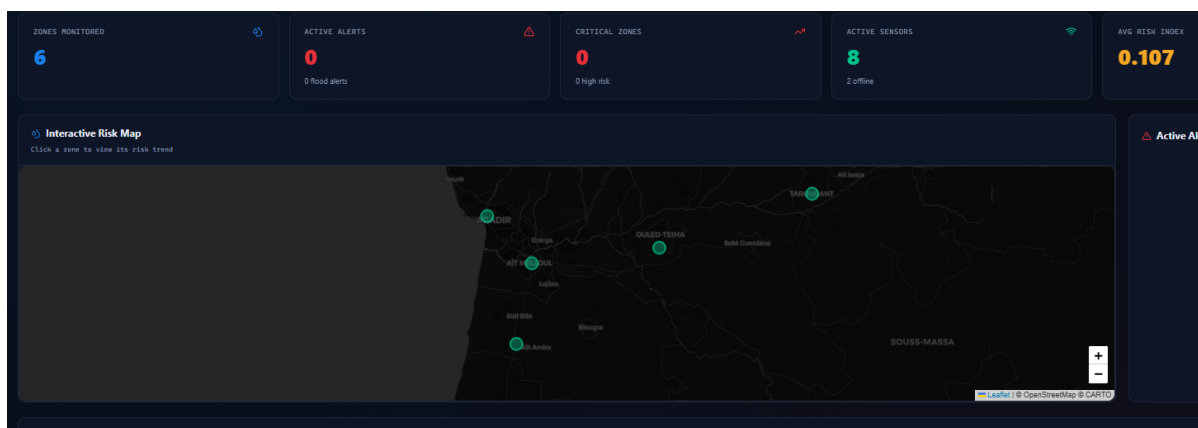


FIGURE 1 – Carte de la région du Souss et emplacement des stations de mesure

Ainsi, ce rapport présente une évaluation approfondie réalisée par l'équipe de sécurité, visant à analyser les mécanismes de protection mis en place au sein de la base de données du projet. L'objectif principal est d'identifier les éventuelles vulnérabilités, notamment celles liées aux attaques courantes telles que les injections SQL, les erreurs de configuration ou encore les failles d'authentification.

Pour mener cette étude, une double approche a été adoptée : une analyse assistée par intelligence artificielle et une analyse manuelle, permettant de comparer l'efficacité, la précision et les limites de chaque méthode. Cette démarche permet non seulement d'améliorer la sécurité du système existant, mais également de proposer des recommandations adaptées aux bonnes pratiques actuelles.

1 Méthodologie de travail

Dans le cadre de l'évaluation de la sécurité du projet Oued Souss Alerte, l'équipe Blue Team a adopté une approche méthodique basée sur plusieurs étapes complémentaires, combinant analyse statique, analyse dynamique et tests de sécurité.

Processus d'évaluation de sécurité - Oued Souss Alerte



FIGURE 2 – Démarche méthodologique de l'audit de sécurité

1.1 Revue du code (Code Review)

La première étape consiste à analyser manuellement le code source de l'application afin d'identifier les vulnérabilités potentielles :

- Vérification de l'utilisation des requêtes préparées (Prepared Statements) pour prévenir les attaques de type injection SQL.
- Identification des concaténations dangereuses dans les requêtes SQL.
- Analyse de la gestion des entrées utilisateur (validation et filtrage).

- Vérification de l'échappement des données affichées (prévention XSS).
- Contrôle de la gestion des erreurs pour éviter la divulgation d'informations sensibles.

1.2 Vérification des mécanismes d'authentification

Cette étape vise à évaluer la robustesse du système d'authentification :

- Vérification de la gestion des sessions (cookies sécurisés, expiration, régénération d'ID de session).
- Contrôle des mécanismes de protection contre les attaques de type brute force (limitation de tentatives).
- Vérification des contrôles d'accès (RBAC – Role-Based Access Control).
- Test de la gestion des privilèges utilisateurs.

1.3 Analyse des risques (Risk Analysis)

Une évaluation des risques a été réalisée afin de prioriser les vulnérabilités identifiées :

- Identification des actifs critiques (base de données, API, comptes utilisateurs).
- Classification des vulnérabilités selon leur gravité (faible, moyenne, élevée, critique).
- Évaluation de l'impact potentiel (perte de données, accès non autorisé, indisponibilité du service).
- Estimation de la probabilité d'exploitation.
- Priorisation des actions correctives selon le niveau de risque.

1.4 Tests d'intrusion internes (Pentesting)

Des tests d'intrusion ont été réalisés en interne pour simuler des attaques réelles sur l'application :

- Utilisation de l'outil OWASP ZAP pour scanner automatiquement les vulnérabilités.
- Détection des failles courantes (SQL Injection, XSS, CSRF, etc.).
- Analyse des réponses du serveur aux requêtes malveillantes.
- Test des endpoints sensibles (formulaires, API, authentification).
- Validation manuelle des vulnérabilités détectées afin d'éviter les faux positifs.

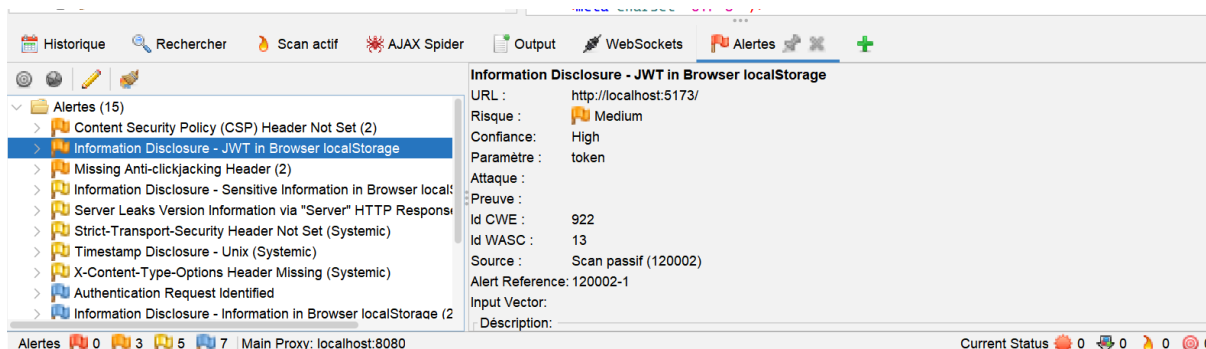


FIGURE 3 – Résultats du scan OWASP ZAP sur l'API

1.5 Corrélation et validation des résultats

Les résultats issus des différentes phases ont été croisés pour garantir leur fiabilité :

- Comparaison entre les vulnérabilités détectées automatiquement et manuellement.
- Confirmation des failles exploitables.
- Documentation détaillée de chaque vulnérabilité (description, impact, preuve).

1.6 Recommandations de sécurité

Enfin, des recommandations ont été formulées pour renforcer la sécurité du système :

- Adoption systématique des bonnes pratiques de développement sécurisé.
- Mise en place de mécanismes de surveillance et de journalisation (logs).
- Mise à jour régulière des dépendances et correctifs de sécurité.
- Formation des développeurs aux pratiques de sécurité.

Version IA

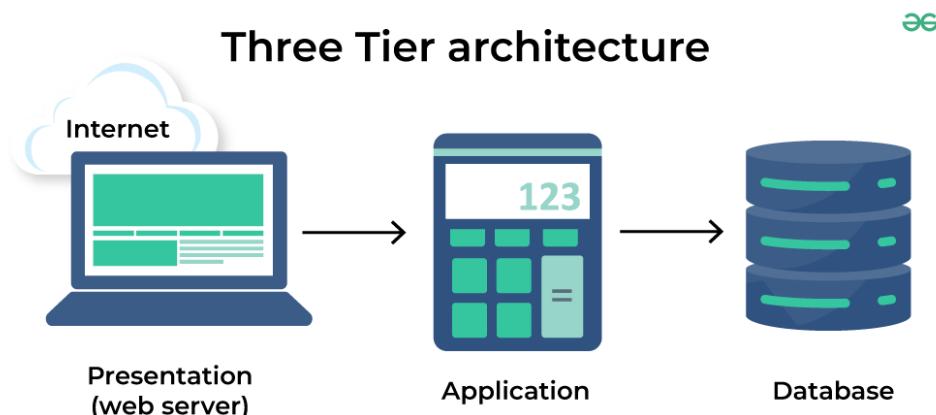


FIGURE 4 – Architecture sécurisée en trois couches identifiée par l'IA

1. Analyse de l'architecture sécurisée

Une architecture en trois couches a été mise en place :

- Frontend (React) avec gestion de l'authentification via JWT.
- Backend (Node.js + Express) intégrant des middlewares de sécurité.
- Base de données (PostgreSQL) avec triggers et procédures stockées.

Cette séparation des couches constitue un bon point de sécurité, car elle limite l'exposition directe de la base de données et permet de centraliser les contrôles au niveau du backend.

2. Identification des mécanismes de sécurité

L'IA a identifié plusieurs mécanismes de sécurité importants :

- **Authentification JWT** : Les utilisateurs sont authentifiés via des tokens expirant après 8 heures, vérifiés à chaque requête, ce qui limite les accès non autorisés.
- **Contrôle d'accès RBAC** : Le système implémente plusieurs rôles (Admin, Opérateur, Lecteur, Sécurité) avec des permissions spécifiques, réduisant les risques d'escalade de privilèges.
- **Protection contre les injections SQL** : L'utilisation de requêtes paramétrées et de validation stricte des entrées est mentionnée.
- **Rate Limiting** : Limitation du nombre de requêtes pour prévenir les attaques brute force ou DoS.
- **Sécurisation des headers HTTP (Helmet)** : Protection XSS, clickjacking, etc.
- **Validation au niveau base de données** : Triggers PostgreSQL.

3. Analyse des bonnes pratiques de développement sécurisé

- Séparation claire entre logique métier, routes et middleware.
- Utilisation d'un système de validation des données en entrée.
- Présence d'un module de sanitization dans les utilitaires backend.
- Tests unitaires et d'intégration couvrant des scénarios de sécurité.

Version manuelle

1. Introduction + résumé exécutif

Contexte analysé

- Backend : API Node.js/Express en TypeScript, stockage MongoDB via Mongoose (`manual_version/b`)
- Frontend : React (Vite) consommant l'API via Axios (`manual_version/frontend/src/...`)

Résumé exécutif (niveau de sécurité)

Niveau global : **Faible à Moyen** (principalement à cause de l'absence de contrôle d'accès).

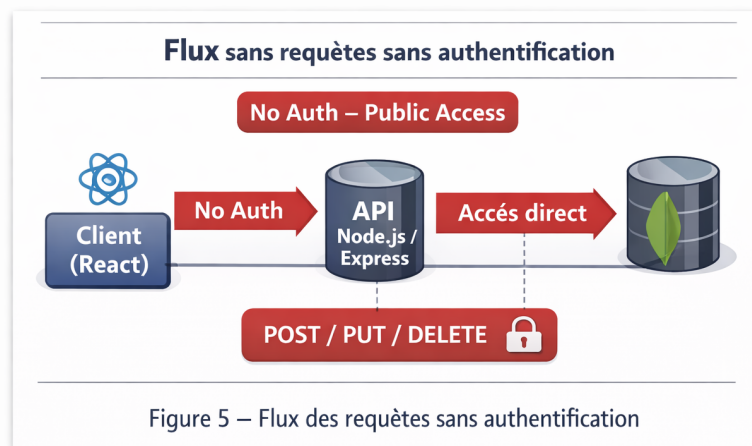


FIGURE 5 – Flux des requêtes sans authentification - accès public aux endpoints critiques

2. Vulnérabilités détectées (priorité critique)

CRITIQUE

VULN-01 — Absence d'authentification / autorisation (Broken Access Control)

Description : aucune vérification d'identité (JWT/session/API key) ni de rôle/-permission. Les routes d'écriture sont accessibles publiquement.

Emplacement :

- `backend/src/server.ts` : aucune couche d'auth globale
- Routes : `zones.ts`, `capteurs.ts`, `mesures.ts`, `alertes.ts` (POST, PUT, DELETE)
- Endpoint admin : `/stats/detailed`

Impact : création/modification/suppression de toutes les données, accès aux statistiques sensibles.

ÉLEVÉ**VULN-02 — CORS permissif**

Description : `app.use(cors())` active un CORS très permissif.

Emplacement : `backend/src/server.ts`

Impact : exposition aux attaques cross-origin, risque CSRF si cookies sont utilisés.

ÉLEVÉ**VULN-03 — Absence de protections baseline**

Description : pas de Helmet, pas de rate limiting, en-tête `X-Powered-By` exposé.

Emplacement : `backend/src/server.ts`, `validate.ts`

Impact : vulnérabilités XSS, clickjacking, DoS, empreinte serveur.

MOYEN**VULN-04 — Journalisation excessive**

Description : en cas d'échec de validation, le code log `body/query/params`.

Emplacement : `backend/src/middleware/validate.ts`

Impact : fuite potentielle de tokens, identifiants, PII dans les logs.

3. Corrections proposées

RECOMMANDATION**CORR-01 — Ajouter une authentification**

Implémenter un middleware de vérification de token JWT ou API Key.

Listing 1 – Middleware d'authentification (`auth.ts`)

```
const requireApiKey = (req, res, next) => {
  const apiKey = req.headers['x-api-key'];
  if (!apiKey || apiKey !== process.env.API_KEY) {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  next();
};
```

RECOMMANDATION**CORR-02 — Restreindre CORS**

```
app.use(cors({
  origin: process.env.CORS_ORIGINS?.split(',') || [],
  credentials: true
}));
```

RECOMMANDATION**CORR-03 — Ajouter Helmet et Rate Limiting**

```
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

app.use(helmet());
app.disable('x-powered-by');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100
});
app.use('/api/', limiter);
```

RECOMMANDATION**CORR-04 — Réduire la verbosité des logs**

```
// AVANT
console.error('Validation failed', { body, query, params });

// APRÈS
console.error('Validation failed', {
  fields: Object.keys(body),
  path: req.path
});
```

4. Tableau comparatif AVANT / APRÈS

Domaine	AVANT	APRÈS (recom-mandé)	Gain
Contrôle d'accès	Aucun	Middleware requireApiKey	Bloque accès anonyme
CORS	<code>cors()</code> sans whitelist	Whitelist CORS_ORIGINS	Réduit exposition
Headers sécurité	Aucun	<code>helmet()</code>	Durcissement
Rate limiting	Aucun	<code>express-rate-limit</code>	Anti-DoS
Logs	Complets	Minimaux	Moins de fuites
Durcissement Express	Defaults	<code>disable('x-powered-by')</code>	Moins d'empainte

TABLE 2 – Comparatif avant/après corrections

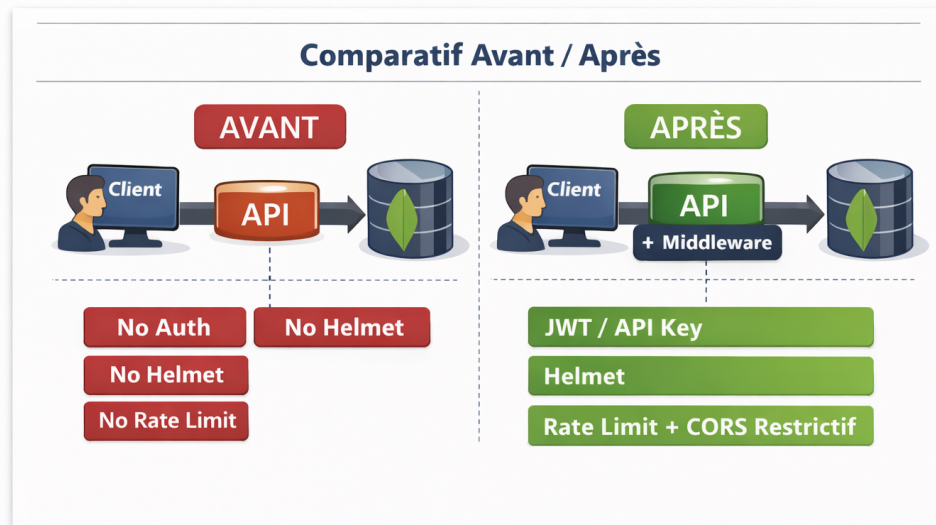


FIGURE 6 – Architecture de sécurité avant et après corrections

5. Bonnes pratiques (OWASP) adaptées au projet

- OWASP A01 : **Broken Access Control** : auth obligatoire, moindre privilège.
- OWASP A02 : **Cryptographic Failures** : secrets via env, rotation, TLS.
- OWASP A03 : **Injection** : validation stricte, whitelist.
- OWASP A05 : **Security Misconfiguration** : Helmet, CORS strict, rate limiting.
- OWASP A09 : **Security Logging** : logs minimisés, corrélation.

6. Conclusion partielle

Le code montre un effort réel sur la validation d'entrées mais le risque majeur est structurel : API ouverte sans authentification.

Comparaison

L'évaluation de la sécurité du projet **Oued Souss Alerte** a été conduite selon deux méthodologies distinctes mais complémentaires.

Critère	Version IA	Version Manuelle
Sécurité perçue	Élevée (JWT, RBAC, Helmet)	Faible à Moyenne (absence d'auth)
Vulnérabilité principale	Aucune	Broken Access Control (Critique)
Faiblesses	Non mentionnées	CORS, headers, rate limiting, logs
Conclusion	Projet conforme aux standards	Risque structurel majeur

TABLE 3 – Comparaison des résultats

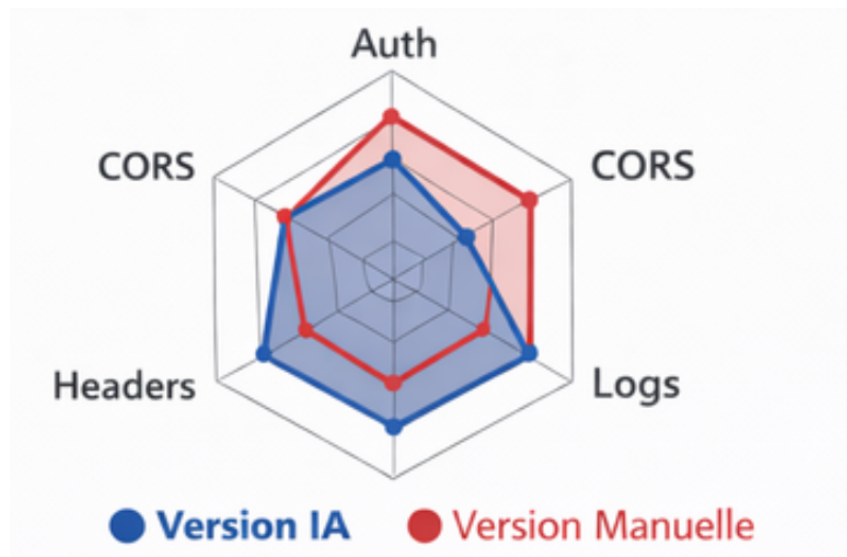


FIGURE 7 – Comparaison radar des scores de sécurité (IA vs Manuel)

Analyse de l'écart

L'écart s'explique par le fossé entre l'architecture documentée et le code déployé.

Conclusion

L'audit de sécurité mené sur le projet **Oued Souss Alerte** a permis de dresser un état des lieux précis.

1. **Fondations solides** : validation des entrées, structuration du code, intentions de sécurisation.
2. **Exposition critique** : faille Broken Access Control majeure.

Recommandation prioritaire

Avant mise en production :

- Implémenter une authentification robuste (JWT ou API Key).
- Durcir la configuration Express (Helmet, Rate Limiting, CORS strict).

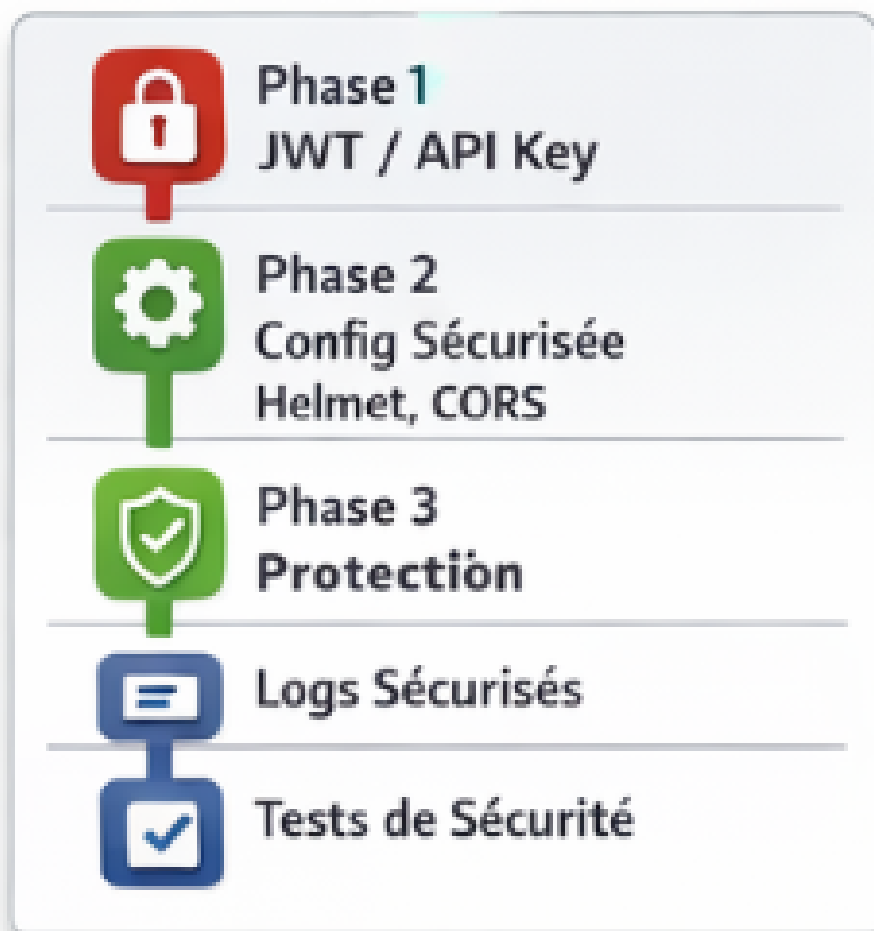


FIGURE 8 – Plan d'action recommandé pour la sécurisation

Bibliographie

1. Outils de test de sécurité (Pentesting)

- OWASP ZAP : Documentation officielle <https://www.zaproxy.org/docs/>
- OWASP ZAP : Guide tutoriel complet <https://www.softwaretestinghelp.com/owasp-zap-tutorial/>
- OWASP ZAP : Guide pratique <https://cloudinsight.cc/en/blog/owasp-zap-tutorial>

2. SIEM et analyse de logs

- Splunk : Documentation officielle <https://help.splunk.com/>
- Splunk : Tutoriel officiel <https://docs.splunk.com/Documentation/Splunk/latest/User/SearchTutorial>